

Metodología I

Unidad 2

Modelos Incrementales y Adaptativos

Nombre: Farias, Gustavo

Comisión: M2025-13

Matrícula: 101662

Repositorio GitHub:

https://github.com/Lucenear/Tecnicatura_Programacion_UTN/tree/main/3er_Semestre/Metodologia_I

Sección A: El Modelo Incremental

1. Por qué el Modelo Incremental es ideal para "CoreBanc 2.0"

Para el reemplazo del sistema central del banco, el modelo incremental es la opción más sensata. Estamos hablando de un sistema "legacy" (heredado) que maneja toda la plata y las cuentas; no podemos permitirnos apagarlo y prender uno nuevo de golpe (el famoso "Big Bang") porque si algo falla, el banco se cae. El modelo incremental nos permite migrar "pieza por pieza". Podemos sacar primero el módulo de consultas de saldo, luego el de transferencias internas, y así sucesivamente. Esto reduce enormemente el riesgo: si hay un error, solo afecta a una parte pequeña y funcional, no a todo el sistema. Además, como la funcionalidad ya existe en el sistema viejo, sabemos qué tenemos que construir, lo que facilita planificar esos incrementos sin sorpresas.

2. Flujo de trabajo en "CoreBanc 2.0"

El flujo sería más o menos así: primero, analizamos el sistema viejo y diseñamos la nueva arquitectura modular. Luego, desarrollamos el primer incremento, que podría ser, por ejemplo, la gestión básica de altas y bajas de clientes (sin movimientos de dinero aún). Este incremento se prueba, se integra y se pone en producción conviviendo con el sistema viejo. Cuando arrancamos con el segundo incremento (digamos, el módulo de depósitos), es crucial hacer pruebas de regresión. Esto significa volver a probar el primer incremento (altas/bajas) para asegurarnos de que el nuevo código de depósitos no rompió nada de lo que ya funcionaba. En un sistema bancario, esto es vital porque todo está conectado; un cambio chico puede tener efectos secundarios graves si no se verifica bien.

3. Diferencia clave: Incremento vs. Prototipo

La diferencia fundamental es que un incremento es software real, productivo y completo en su funcionalidad parcial. Cuando entrego el incremento de "consultas de saldo", el usuario puede usarlo de verdad para ver su plata. Es una parte final del producto.

En cambio, un prototipo (como vimos en el TP anterior) suele ser desechable o incompleto; sirve para validar una idea o una interfaz, pero no está pensado para estar en producción ni manejar datos reales de forma permanente. En "CoreBanc 2.0" no estamos probando ideas, estamos construyendo el sistema final bloque por bloque.

Sección B: Desarrollo Ágil

1. Por qué Ágil es la mejor opción para "ZenPay"

Para la nueva app FinTech "ZenPay", un enfoque rígido no sirve de nada. El mercado de las fintech cambia semana a semana y los requisitos son desconocidos. Aquí necesitamos velocidad (Time-to-Market) y capacidad de reacción. Los modelos clásicos sufren porque intentan predecir el futuro; Ágil, en cambio, asume que vamos a equivocarnos al principio y nos prepara para corregir el rumbo rápido. Al trabajar de forma ágil, podemos lanzar una versión mínima, ver cómo reacciona la gente, y cambiar las prioridades para el siguiente mes. Esto mitiga el riesgo de gastar millones desarrollando una app que nadie quiere.

2. Valores del Manifiesto Ágil aplicados a "ZenPay"

Dos valores son claves acá:

- "Respuesta al cambio sobre seguir un plan": En ZenPay, si sale una nueva tecnología de pagos o un competidor lanza una función copada, tenemos que poder incorporar eso ya, no esperar seis meses a que termine la "fase de diseño". El plan inicial va a cambiar seguro, y está bien.
- "Colaboración con el cliente sobre negociación contractual": El Product Owner del banco tiene que estar metido en el equipo día a día. No sirve que nos entregue un documento firmado y desaparezca. Necesitamos su feedback constante para saber si la funcionalidad de "micro-inversiones" está yendo por buen camino o si los usuarios la encuentran confusa. Esa comunicación directa vale más que cualquier contrato detallado al inicio.

3. Scrum y XP en acción

Para gestionar ZenPay, usaría Scrum para organizar el trabajo: tendríamos un Product Owner que define qué es lo más urgente (el backlog), y el equipo trabajaría en Sprints de 2 o 3 semanas. Al final de cada Sprint, entregamos algo funcional para mostrarle al stakeholders.

Para asegurar que el código sea de calidad (porque es una app financiera y tiene que ser segura y rápida), aplicaría prácticas de Extreme Programming (XP):

- Programación en Pareja (Pair Programming): Dos desarrolladores trabajando juntos en la misma pantalla. Esto mejora la calidad del código al instante y comparte conocimiento, evitando que alguien quede como "único dueño" de una parte crítica.
- Integración Continua: Cada vez que un programador termina una tarea, su código se integra con el del resto y se ejecutan pruebas automáticas. Así detectamos errores de compatibilidad al momento, no al final del proyecto.

Sección C: Comparación y Conclusión

1. Predictivo vs. Adaptativo

- Proyecto A (CoreBanc 2.0): Es de filosofía Predictiva. Aunque usamos incrementos, sabemos qué hay que construir porque el sistema viejo ya existe. La planificación es más rígida porque no podemos improvisar con el dinero de la gente. Sabemos el destino, solo tenemos que llegar ahí con cuidado.
- Proyecto B (ZenPay): Es claramente Adaptativo. No sabemos exactamente qué va a funcionar en el mercado. La planificación es flexible y se ajusta sprint a sprint según lo que aprendemos. El objetivo es descubrir el producto mientras lo construimos.

2. La analogía: Mapa vs. Brújula

- Para CoreBanc 2.0, recomendaría usar un "mapa detallado". Ya conocemos el terreno (el sistema legacy), sabemos dónde están los obstáculos y tenemos una ruta clara de migración. Necesitamos ese mapa para no perdernos y asegurar que cada pieza encaje en el lugar correcto.

- Para ZenPay, necesitamos una "brújula". No hay mapa porque estamos explorando un territorio nuevo (el mercado fintech actual). La brújula nos marca la dirección general (crear una app competitiva), pero nos permite cambiar el camino si encontramos un río o una montaña inesperada (feedback de usuarios o cambios del mercado).

3. Recomendación Final para la Dirección del Banco

Estimados directivos, para garantizar el éxito de ambos proyectos, es crucial no aplicar la misma metodología a los dos. Para "CoreBanc 2.0", deben adoptar el Modelo Incremental, ya que su prioridad es la seguridad y la continuidad operativa; este enfoque les permite modernizar el sistema crítico pieza por pieza, minimizando el riesgo de caída del servicio y permitiendo una migración controlada. Por otro lado, para "ZenPay", deben implementar un marco Ágil (Scrum/XP), porque el riesgo aquí es comercial, no técnico; necesitan lanzar rápido, aprender de los usuarios y adaptar la app constantemente para competir con las startups. Usar Ágil aquí les da la velocidad y flexibilidad que el mercado exige, mientras que el enfoque incremental en el core bancario les da la estabilidad que la operación requiere.